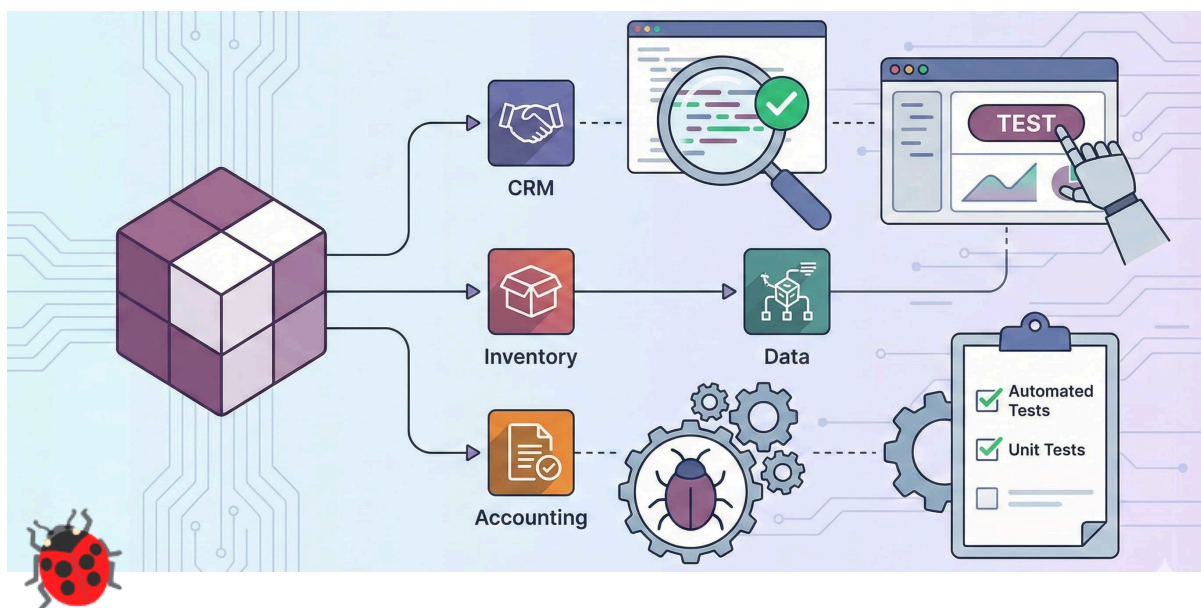




GUÍA COMPLETA DE PRUEBAS PARA MÓDULOS DE ODOO

Sunname Technologies

Manual para Consultores y Desarrolladores

Versión 1.0 - Febrero 2026

 [Documentación Integral de Testing para Desarrolladores](#)

Introducción

Este documento ha sido creado específicamente para **Sunname Technologies** con el objetivo de establecer un protocolo robusto de pruebas para módulos de Odoo desarrollados por terceros. Como empresa joven en crecimiento, es fundamental que podamos verificar la calidad de los desarrollos que recibimos antes de implementarlos en ambientes de producción.

¿Por qué este documento?

Hemos identificado un patrón recurrente: los módulos que recibimos funcionan bien inicialmente, pero después de un tiempo empiezan a presentar problemas

que afectan a otros módulos o funcionalidades del sistema. Esto sucede porque:

- No siempre se prueban todas las interacciones posibles con otros módulos instalados
- Los desarrolladores externos no conocen todo nuestro ecosistema de módulos personalizados
- Falta un protocolo estandarizado de pruebas de calidad
- No hay suficiente documentación sobre casos de uso específicos de nuestros clientes
- Los errores solo aparecen cuando hay carga real de datos o usuarios simultáneos

Estructura del documento

Este manual está dividido en dos grandes secciones:

- **Sección para Consultores No Técnicos:** Enfocada en pruebas funcionales que no requieren conocimientos de programación
- **Sección para Desarrolladores:** Centrada en aspectos técnicos, revisión de código y pruebas automatizadas

Ambos roles son fundamentales para garantizar la calidad de los módulos que implementamos. Este documento debe ser una referencia constante en nuestros procesos de validación.

PARTE 1: GUÍA PARA CONSULTORES NO TÉCNICOS

Como consultor, eres el primer filtro de calidad. Tu rol es fundamental porque conoces las necesidades del negocio, los procesos de los clientes y cómo debería funcionar el sistema desde la perspectiva del usuario final. No necesitas ser programador para hacer estas pruebas.

1. Pruebas Funcionales Básicas

¿Qué son y por qué hacerlas?

Las pruebas funcionales verifican que el módulo hace exactamente lo que se supone debe hacer según los requerimientos. Es como probar que un carro arranca, frena y acelera correctamente antes de venderlo.

¿Cuándo hacerlas? Inmediatamente después de recibir el módulo, antes de cualquier otra prueba.

Cómo hacer las pruebas

Paso 1: Revisión de Documentación

- Pedir al desarrollador un documento con: descripción del módulo, qué problema resuelve, y lista de funcionalidades incluidas
- Verificar que tenga capturas de pantalla o videos de las funcionalidades principales
- Confirmar que incluya instrucciones de configuración inicial si las hay



Ejemplo práctico:

Si el módulo es para 'Gestión de Comisiones de Vendedores', debe explicar: cómo se calculan las comisiones, qué criterios se usan, dónde se configuran las reglas, y cómo los vendedores pueden ver sus comisiones.

Paso 2: Instalación en Ambiente de Pruebas

- **NUNCA instalar directamente en producción** - siempre usar una copia de la base de datos
- Anotar cualquier mensaje de error o advertencia durante la instalación
- Verificar que todos los menús y opciones aparezcan donde deberían



Señal de alerta:

Si durante la instalación aparece algún error (aunque el módulo se instale igual), detén las pruebas y reporta al desarrollador. Esto indica problemas de dependencias o configuración.

Paso 3: Prueba del Flujo Principal (Camino Feliz)

El 'camino feliz' es cuando todo funciona perfectamente, sin errores ni situaciones extrañas. Es lo que el desarrollador probablemente ya probó.

- Crear un caso de uso simple y directo
- Seguir paso a paso el proceso principal
- Verificar que el resultado final sea el esperado
- Documentar con capturas de pantalla cada paso



Ejemplo práctico:

Módulo de comisiones: Crear un vendedor, configurar su regla de comisión (ej: 5% de ventas), crear una venta, confirmarla, y verificar que se calcule y muestre la comisión correcta.

2. Pruebas de Casos Extremos (Edge Cases)

Los casos extremos son situaciones inusuales o límite que pueden romper el sistema. Los desarrolladores a veces no los consideran porque piensan 'nadie haría eso'. Pero en la vida real, los usuarios hacen cosas inesperadas todo el tiempo.

¿Qué son y por qué hacerlas?

¿Cuándo hacerlas? Después de confirmar que el flujo principal funciona.

Tipos de casos extremos a probar

Tipo de Caso	Qué Probar
Campos Vacíos	Qué pasa si dejas campos importantes vacíos o con espacios en blanco
Valores Cero o Negativos	Intentar poner 0, -1, o números muy grandes en campos numéricos
Texto Muy Largo	Escribir textos extremadamente largos en campos de texto
Caracteres Especiales	Usar ñ, acentos, símbolos raros: @\$%&, emojis 😊
Fechas Inválidas	Fechas en el pasado, futuro lejano, 29 de febrero en años no bisiestos

Tipo de Caso	Qué Probar
Borrar en Medio del Proceso	Empezar un proceso, borrarlo, y ver si quedaron datos 'huérfanos'



Ejemplo práctico completo

Módulo de comisiones - casos extremos a probar:

1. ¿Qué pasa si configuro una comisión de 0%?
2. ¿Y si pongo 150% (más del 100%)?
3. ¿Funciona con ventas con descuento del 100% (gratis)?
4. ¿Qué pasa si cancelo una venta que ya generó comisión?
5. ¿Puedo tener dos vendedores en la misma venta? ¿Cómo se divide la comisión?
6. ¿Funciona si el vendedor no tiene configurada ninguna regla de comisión?

3. Pruebas de Permisos y Seguridad

¿Qué son y por qué hacerlas?

Odoo tiene un sistema robusto de permisos. Si el módulo nuevo no respeta esto, un empleado podría ver o modificar información que no debería. Esto es crítico para la seguridad de los datos.

¿Cuándo hacerlas? Después de las pruebas funcionales básicas.

Haz una lista de los diferentes tipos de usuario que interactuarán con el módulo:

Cómo hacer las pruebas

Paso 1: Identificar los roles de usuario

- Usuario básico (puede ver pero no modificar)
- Usuario avanzado (puede crear y modificar)
- Manager (puede validar, cancelar, borrar)

- Administrador (acceso total)
- Usuario de otro departamento (no debería ver nada de este módulo)

Paso 2: Crear usuarios de prueba

- En tu ambiente de pruebas, crea un usuario para cada rol identificado
- Asigna los permisos correspondientes de Odoo (Configuración > Usuarios y Compañías > Usuarios)
- Anota las contraseñas para poder entrar como cada usuario

Paso 3: Probar con cada rol

Para cada usuario, verifica:

- ¿Puede ver el menú del módulo? (debería verlo solo si tiene permisos)
- ¿Puede crear nuevos registros? (solo si tiene permiso de creación)
- ¿Puede modificar registros existentes?
- ¿Puede borrar registros?

Ejemplo práctico:

- ¿Puede ver todos los registros o solo los suyos?



Módulo de comisiones: *Un vendedor solo debería ver SUS comisiones, no las de otros vendedores. El gerente de ventas debería ver todas. Un usuario de contabilidad tal vez pueda verlas pero no modificarlas.*



Señal de alerta:

Si un usuario sin permisos puede ver o modificar datos que no debería, esto es un problema de seguridad GRAVE. Debe reportarse inmediatamente.

4. Pruebas de Compatibilidad con Otros Módulos

¿Qué son y por qué hacerlas?

ESTA ES LA PRUEBA MÁS IMPORTANTE PARA USTEDES. Según mencionaste, aquí es donde aparecen la mayoría de sus problemas. Un módulo puede

funcionar perfectamente solo, pero cuando interactúa con otros módulos instalados, empiezan los conflictos.

¿Cuándo hacerlas? Después de confirmar que el módulo funciona bien por sí solo.

Cómo hacer las pruebas

Paso 1: Hacer un inventario de módulos relacionados

Identifica qué módulos ya instalados podrían interactuar con el nuevo:

- Módulos que usan las mismas tablas de datos (ej: ventas, productos, clientes)
- Módulos que modifican los mismos formularios o vistas
- Módulos que tienen flujos de trabajo similares
- Módulos personalizados anteriores del mismo desarrollador



Ejemplo:

Si el módulo nuevo es de 'Comisiones', debe funcionar con: módulo de Ventas (standard), módulo de CRM si lo usan, cualquier módulo personalizado de precios o descuentos, módulo de reportes de ventas si existe.

Paso 2: Probar flujos de trabajo cruzados

Esta es la parte que más tiempo toma pero es la más valiosa:

- Ejecuta el proceso completo usando AMBOS módulos al mismo tiempo
- Alterna entre usar primero uno y luego el otro
- Busca inconsistencias en los datos
- Verifica que los reportes incluyan datos de ambos módulos correctamente



Ejemplo práctico:

Tienes módulo de Comisiones (nuevo) + Módulo de Descuentos Especiales (ya instalado):

1. Crea una venta normal - verifica que la comisión se calcule bien
2. Crea una venta CON descuento especial - ¿la comisión se calcula sobre el precio con descuento o sin descuento?
3. Aplica el descuento DESPUÉS de crear la venta - ¿se recalcula la comisión automáticamente?
4. Revisa que ambos módulos guarden sus datos correctamente sin sobrescribirse

Paso 3: Revisar vistas y formularios compartidos

Si dos módulos modifican el mismo formulario (ej: el formulario de Venta):

- Abre el formulario y verifica que TODOS los campos de ambos módulos aparezcan
- Verifica que no haya campos duplicados
- Asegúrate que el formulario no se vea 'roto' o mal acomodado
- Confirma que todos los botones funcionen correctamente



Señal de alerta:

Si un campo desaparece, aparece duplicado, o ves errores de formato en formularios que antes funcionaban bien, hay un conflicto entre módulos.

5. Pruebas de Rendimiento Básicas

¿Qué son y por qué hacerlas?

Un módulo puede funcionar bien con 5 registros, pero cuando tienes 5,000 puede volverse muy lento o incluso dar errores de tiempo de espera. Como consultor, puedes hacer pruebas básicas de rendimiento sin ser técnico.

¿Cuándo hacerlas? Después de las pruebas funcionales y antes de ir a producción.

Cómo hacer las pruebas

Paso 1: Crear datos de prueba realistas

No pruebes solo con 2 o 3 registros. Necesitas volumen:

- Si el cliente tiene 100 productos, crea 100 productos de prueba
- Si procesa 50 ventas al día, crea al menos 500 ventas de prueba
- Usa herramientas como Excel para generar datos masivos y luego importarlos



Tip práctico:

Si puedes, copia datos reales anonimizados de producción a tu ambiente de pruebas. Esto te da el volumen y complejidad real.

Paso 2: Probar la carga de vistas y reportes

Con tus datos de prueba cargados:

- Abre las vistas de lista - ¿tardan más de 3-5 segundos en cargar?
- Genera reportes - ¿cuánto tiempo toman? (más de 30 segundos es preocupante)
- Usa los filtros de búsqueda - ¿responden rápido?
- Exporta datos a Excel - ¿funciona sin problemas con muchos registros?

¿Qué es tiempo aceptable?

Acción	Tiempo Bueno	Tiempo Preocupante
Abrir lista	< 3 segundos	> 8 segundos
Abrir formulario	< 2 segundos	> 5 segundos
Guardar registro	< 1 segundo	> 3 segundos
Generar reporte	< 10 segundos	> 30 segundos
Buscar/Filtrar	< 2 segundos	> 5 segundos

6. Pruebas de Validación de Datos

¿Qué son y por qué hacerlas?

El sistema debe protegerte de cometer errores. Si puedes guardar datos incorrectos o ilógicos, eventualmente esto causará problemas. Las validaciones son como los cinturones de seguridad del sistema.

¿Cuándo hacerlas? Durante las pruebas funcionales, específicamente probando escenarios incorrectos.

Qué validaciones debe tener el módulo

1. Validaciones de Campos Obligatorios

- Intenta guardar sin llenar campos importantes
- El sistema debe mostrar un mensaje claro indicando qué falta
- El mensaje debe estar en español (si es tu idioma) y ser entendible



Ejemplo:

Si intentas crear una comisión sin especificar el vendedor, debe decir: 'El campo Vendedor es obligatorio' (no solo 'Error' o un mensaje técnico confuso).

2. Validaciones de Lógica de Negocio

Más allá de campos obligatorios, debe validar que los datos tengan sentido:

- Fechas: La fecha de fin no puede ser antes que la de inicio
- Números: No puedes tener cantidades negativas donde no tiene sentido
- Referencias: No puedes hacer referencia a algo que no existe
- Duplicados: Evitar crear duplicados donde no deberían existir



Ejemplo:

No deberías poder configurar dos reglas de comisión activas al mismo tiempo para el mismo vendedor. El sistema debe advertir: 'Este vendedor ya tiene una regla de comisión activa'.

3. Validaciones de Formato

- Emails: Debe validar que tenga formato de email (@, punto, etc.)
- Teléfonos: Debe aceptar diferentes formatos pero validar que sean números
- Códigos: Si hay formato específico (ej: RFC), debe validarlo

7. Pruebas de Idioma y Traducciones

¿Qué son y por qué hacerlas?

Si trabajan con clientes en diferentes idiomas, o simplemente quieren que todo esté en español, es importante verificar que todas las etiquetas, mensajes y reportes estén en el idioma correcto.

¿Cuándo hacerlas? Una vez que el módulo funciona correctamente.

Qué revisar

- **Etiquetas de campos:** Todos los nombres de campos deben estar traducidos
- **Mensajes de error:** Deben estar en español, no en inglés técnico
- **Títulos de menús:** Verificar en la navegación principal
- **Reportes:** Encabezados, columnas y pie de página
- **Botones:** 'Guardar', 'Cancelar', 'Confirmar', etc.



Señal de alerta:

Si encuentras texto en inglés donde debería estar en español, especialmente en mensajes de error o etiquetas de campos, es señal de que el módulo no está completamente terminado.

8. Pruebas de Documentación y Usabilidad

¿Qué son y por qué hacerlas?

Un módulo puede ser técnicamente perfecto, pero si es confuso de usar o no tiene documentación, tus clientes tendrán problemas. La usabilidad determina si el módulo realmente agrega valor o genera frustración.

¿Cuándo hacerlas? Durante todo el proceso de pruebas, con la perspectiva de un usuario nuevo.

Lista de verificación de usabilidad

Aspecto a Revisar	Qué Verificar
Navegación Intuitiva	¿Es fácil encontrar las opciones? ¿Los menús tienen nombres claros?
Tooltips y Ayuda	¿Hay textos de ayuda cuando pasas el mouse sobre campos complejos?
Mensajes Claros	¿Los mensajes de confirmación/error son comprensibles?
Flujo Lógico	¿El proceso sigue un orden natural? ¿O hay que ir y venir?
Valores por Defecto	¿Los campos tienen valores predeterminados sensatos?
Manual de Usuario	¿Existe? ¿Es claro? ¿Tiene capturas actualizadas?



Ejercicio práctico:

Imagina que eres un empleado nuevo que nunca ha visto este módulo. ¿Podrías usarlo sin que nadie te explique? Si la respuesta es no, hay un problema de usabilidad.

Checklist Final para Consultores

Antes de dar tu aprobación, verifica que hayas completado:

✓	Prueba Realizada
☐	Revisión de documentación completa
☐	Instalación exitosa sin errores
☐	Flujo principal funciona correctamente
☐	Al menos 5 casos extremos probados
☐	Permisos verificados para 3+ roles diferentes
☐	Compatibilidad con módulos existentes confirmada

✓	Prueba Realizada
☐	Pruebas con volumen realista de datos
☐	Validaciones de datos funcionando
☐	Traducciones completas
☐	Usabilidad aceptable para usuarios finales
☐	Documentación clara y disponible

PARTE 2: GUÍA PARA DESARROLLADORES

Como desarrollador, tu responsabilidad va más allá de verificar que funcione. Debes revisar la calidad del código, la arquitectura, el rendimiento técnico y la mantenibilidad a largo plazo. Estas son pruebas que los consultores no pueden hacer.

1. Revisión de Código (Code Review)

¿Qué es y por qué hacerla?

La revisión de código es el **análisis del código fuente del módulo** para identificar problemas de calidad, seguridad, mantenibilidad y adherencia a las mejores prácticas de Odoo.

Es como una inspección de los cimientos de un edificio antes de habitarlo.

¿Por qué es crítico? El código mal escrito puede funcionar ahora pero causar problemas imposibles de debuggear después, especialmente con actualizaciones de Odoo o cuando se instalan más módulos.

¿Cuándo hacerla? Antes de cualquier prueba funcional. Si el código es malo, no vale la pena probar el resto.

Aspectos a revisar

1. Estructura del Módulo

Verifica que el módulo siga la estructura estándar de Odoo:

Estructura correcta:

```
module_name/
├── __init__.py
```

```
|— __manifest__.py
|— models/
|   |— __init__.py
|   └─ *.py
|— views/
|   └─ *.xml
|— security/
|   |— ir.model.access.csv
|   └─ security.xml
|— data/
|   └─ *.xml
|— static/
|   └─ description/
|       └─ icon.png
└─ README.md
```



Señales de alerta:

- Archivos sueltos en el directorio raíz que no deberían estar ahí
- Falta el archivo `__manifest__.py` o tiene información incompleta
- No hay separación clara entre modelos, vistas y seguridad
- Archivos `.pyc` o `__pycache__` incluidos en el módulo

2. Archivo `__manifest__.py`

Revisar que contenga toda la información necesaria:



Ejemplo de manifest correcto:

```
{
    'name': 'Gestión de Comisiones',
    'version': '16.0.1.0.0', # Versión Odoo.mayor.
menor.parche
    'category': 'Sales',
    'summary': 'Cálculo automático de comisiones pa
ra vendedores',
    'description': '''Descripción detallada del mód
ulo''',
    'author': 'Nombre del Desarrollador/Empresa',
    'website': 'https://...',
    'depends': ['sale', 'account'], # Dependencias
explícitas
    'data': [
        'security/ir.model.access.csv',
        'views/commission_views.xml',
        'reports/commission_report.xml',
    ],
    'demo': [],
    'installable': True,
    'application': False,
    'auto_install': False,
    'license': 'LGPL-3',
}
```

Qué verificar:

- **version:** Debe empezar con la versión de Odoo (ej: 16.0, 17.0)
- **depends:** TODAS las dependencias deben estar listadas. Si falta alguna, el módulo puede fallar en instalaciones limpias
- **data:** El orden importa. Seguridad primero, luego datos, luego vistas
- **license:** Debe ser compatible con Odoo (LGPL-3 o superior)

3. Código de Modelos (models/*.py)

a) Herencia Correcta

✓ Correcto - Herencia por Extensión (PREFERIDA):

```
class SaleOrder(models.Model):
    _inherit = 'sale.order' # Extiende el modelo existente

    commission_amount = fields.Float(
        string='Comisión',
        compute='_compute_commission',
        store=True
    )
```

Analogía

✗ Incorrecto - Sobrescribir métodos sin llamar super():

```
def action_confirm(self):
    # MALO: No llama super(), borra funcionalidad existente
    self.state = 'sale'
    return True
```

Analogía

✓ Correcto - Con super():

```
def action_confirm(self):
    # BIEN: Mantiene funcionalidad original
    res = super(SaleOrder, self).action_confirm()
    # Tu código adicional aquí
    self._calculate_commission()
    return res
```

Analogía

b) Campos Computados Correctamente

✓ Correcto:

```
commission_amount = fields.Float(  
    compute='_compute_commission',  
    store=True,      # Almacena en BD para performance  
    readonly=True    # El usuario no debe editarlo  
)  
  
@api.depends('order_line.price_subtotal', 'commission_r  
ate')  
def _compute_commission(self):  
    for order in self:  
        order.commission_amount = \  
            order.amount_untaxed * (order.commission_ra  
te / 100.0)
```

Analogía

✗ Errores comunes:

- Campo computado sin `@api.depends` (no se recalcula automáticamente)
- Falta `store=True` cuando el valor se usa en búsquedas o reportes
- Usar `self` sin `for loop` (falla con múltiples registros)
- Operaciones pesadas en compute sin considerar performance

c) Constraints y Validaciones

✓ Correcto:

```
@api.constrains('commission_rate')  
def _check_commission_rate(self):  
    for record in self:  
        if record.commission_rate < 0 or record.commiss  
ion_rate > 100:  
            raise ValidationError(  

```

```
        'La tasa de comisión debe estar entre 0  
y 100'  
    )
```

Analogía

d) Nombres y Convenciones

- Nombres de campos: `snake_case` (commission_amount, not commissionAmount)
- Nombres de clases: `PascalCase` (SaleOrder, not sale_order)
- Métodos privados: empiezan con `_` (`_compute_commission`)
- Traducibles: usar `_()` para textos visibles al usuario

4. Archivos de Seguridad

Verificar ir.model.access.csv:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink  
access_commission_user,commission.user,model_sale_commission,base.group_user,1,0,0,0  
access_commission_manager,commission.manager,model_sale_commission,sales_team.group_sale_manager,1,1,1,1
```

Qué verificar:

- TODOS los modelos nuevos deben tener permisos definidos
- Permisos asignados a grupos existentes de Odoo
- Principio de mínimo privilegio (no dar permisos innecesarios)
- Separar permisos de usuario básico vs manager



Señal de alerta:

Si el archivo ir.model.access.csv está vacío o falta, NADIE podrá acceder al módulo excepto administradores. Esto es un error crítico.

2. Pruebas de Integración con Otros Módulos

¿Qué son y por qué hacerlas?

Las pruebas de integración verifican que el módulo funcione correctamente con otros módulos instalados. A nivel técnico, esto significa revisar herencias, sobrescrituras de métodos, y posibles conflictos de XML.

¿Por qué es crítico? Este es su principal problema según lo que me compartiste. Módulos que funcionan solos pero fallan con otros.

¿Cuándo hacerlas? Después de la revisión de código y antes del despliegue.

Tipos de conflictos a detectar

1. Conflictos de Vistas XML

Problema común: Dos módulos intentan modificar la misma vista de forma incompatible.

Cómo detectarlo:

1. Revisar el log de Odoo durante la instalación
2. Abrir las vistas afectadas y verificar que todos los campos aparezcan
 - a. Usar el modo desarrollador para inspeccionar la vista compilada



Ejemplo de conflicto:

Módulo A agrega campos después de 'partner_id' Módulo B también agrega campos después de 'partner_id' Resultado: Los campos pueden aparecer en orden inesperado o uno puede sobrescribir al otro

Solución recomendada:

Usar `xpath` con posiciones relativas claras y prioridades en las vistas

2. Conflictos de Métodos Sobreescritos

Problema común: Múltiples módulos sobrescriben el mismo método sin coordinar.



Ejemplo de problema:

```
# Módulo A
def action_confirm(self):
    res = super().action_confirm()
    self._do_something_A()
    return res

# Módulo B (instalado después)
def action_confirm(self):
    res = super().action_confirm()
    self._do_something_B()
    return res
# ✅ BIEN: Ambos se ejecutan

# Módulo C (mal hecho)
def action_confirm(self):
    # ❌ MAL: No llama super(), rompe A y B
    self._do_something_C()
    self.state = 'sale'
    return True
```

Explicación

Cómo detectarlo:

- Buscar en el código todos los métodos que se sobrescriben
- Verificar que SIEMPRE llamen `super()`
- Probar el flujo completo con todos los módulos activos
- Revisar logs por errores de `AttributeError` o llamadas a métodos inexistentes

3. Conflictos de Dependencias

Cómo verificar dependencias:

1. Crear una base de datos limpia de prueba
2. Instalar SOLO las dependencias listadas en `__manifest__.py`
3. Intentar instalar el módulo nuevo
4. Si falla, significa que faltan dependencias en el manifest

**Señal de alerta:**

Errores como

- ***Model sale.order.commission not found***
- ***View sale.view_order_form does not exist***

indican dependencias faltantes.

3. Pruebas Unitarias Automatizadas

¿Qué son y por qué hacerlas?

Las pruebas unitarias son scripts de Python que verifican automáticamente que cada función del módulo hace lo que debe hacer. Es como tener un robot que prueba el módulo miles de veces en segundos.

¿Por qué es crítico? Permiten detectar regresiones rápidamente. Si el desarrollador hace cambios en el futuro, las pruebas avisan si algo se rompió.

¿Cuándo exigirlos? Para módulos complejos o críticos, especialmente si planean mantenerlos a largo plazo.

Qué debe incluir el módulo

El módulo debería tener un directorio `tests/` con pruebas:

```
module_name/  
└─ tests/  
    ├── __init__.py  
    ├── test_commission_calculation.py  
    └─ test_commission_security.py
```



Ejemplo de prueba básica:

```
from odoo.tests import TransactionCase

class TestCommissionCalculation(TransactionCase):

    def setUp(self):
        super().setUp()
        self.SaleOrder = self.env['sale.order']
        self.partner = self.env.ref('base.res_partner_1')

    def test_commission_basic_calculation(self):
        """Test: Comisión se calcula correctamente con tasa del 10%"""
        order = self.SaleOrder.create({
            'partner_id': self.partner.id,
            'commission_rate': 10.0,
        })

        # Agregar línea de $1000
        order.order_line = [(0, 0, {
            'product_id': self.env.ref('product.product_product_1').id,
            'product_uom_qty': 1,
            'price_unit': 1000.0,
        })]

        # Verificar cálculo
        self.assertEqual(order.commission_amount, 100.0, 'Comisión debería ser $100 (10% de $1000)')

    def test_commission_with_zero_rate(self):
        """Test: Comisión con tasa 0% debe ser 0"""
        order = self.SaleOrder.create({
            'partner_id': self.partner.id,
            'commission_rate': 0.0,
        })
```

```
        order.order_line = [(0, 0, {
            'product_id': self.env.ref('product.pro
duct_product_1').id,
            'product_uom_qty': 1,
            'price_unit': 1000.0,
        })]

        self.assertEqual(order.commission_amount,
0.0)
```

Explicación

Cómo ejecutar las pruebas:

```
# En la línea de comandos:
odoo-bin -c odoo.conf -d test_database -i module_name --tes
t-enable --stop-after-init

# O solo las pruebas del módulo:
odoo-bin -c odoo.conf -d test_database --test-tags module_n
ame
```

Qué verificar en los resultados:

- Todas las pruebas deben pasar (100% success rate)
- No debe haber warnings relacionados con el módulo
- Las pruebas deben cubrir los casos principales y casos extremos



Nota importante:

Si el desarrollador no incluyó pruebas, puedes pedirle que las agregue o escribirlas ustedes mismos. Es una inversión que vale la pena para módulos complejos.

4. Pruebas de Performance y Optimización

¿Qué son y por qué hacerlas?

A nivel técnico, estas pruebas identifican cuellos de botella en el código: consultas SQL ineficientes, loops innecesarios, campos computados sin optimizar, etc.

¿Por qué es crítico? Un módulo lento puede hacer que TODO Odoo se vuelva lento, afectando la experiencia de todos los usuarios.

¿Cuándo hacerlas? Después de confirmar que el módulo funciona correctamente.

Herramientas para analizar performance

1. Modo Debug con SQL Profiling

Odoo tiene un modo debug que muestra cuántas consultas SQL se ejecutan:

1. Activar modo desarrollador
2. Agregar ?debug=assets a la URL
3. Ejecutar la acción a medir (ej: abrir una vista, generar reporte)
4. Ver el contador de queries en la esquina superior derecha

Qué buscar:

Acción	Queries Aceptables	Queries Preocupantes
Abrir lista de 20 ítems	< 50 queries	> 200 queries
Abrir formulario	< 30 queries	> 100 queries
Guardar registro	< 20 queries	> 50 queries
Generar reporte simple	< 100 queries	> 500 queries

2. Log de Odoo para Queries Lentas

Configurar odoo.conf para logear queries lentas:

```
[options]
log_level = debug_sql
log_db_level = warning
pg_path = /usr/lib/postgresql/12/bin
```


Esto mostrará en el log todas las queries que tomen más de 1 segundo.

3. Análisis de Código

Buscar patrones problemáticos en el código:

❌ Patrón 1: Loops con búsquedas individuales

```
# MALO: N+1 queries problem
for order in orders:
    partner = self.env['res.partner'].search([('id', '=', order.partner_id.id)])
    # Hace una query por cada orden!
```

Explicación

✅ Mejor: Una query para todo

```
# BIEN: Una sola query
partner_ids = orders.mapped('partner_id.id')
partners = self.env['res.partner'].browse(partner_ids)
```

Explicación

❌ Patrón 2: Campos computados sin store=True

```
# MALO: Campo usado en búsquedas pero no almacenado
total_commission = fields.Float(
    compute='_compute_total',
    # Falta store=True!
)

# Cada búsqueda requiere recalcular TODOS los registros
records = self.env['sale.commission'].search(
    [('total_commission', '>', 1000)]
```

```
)  
# MUY LENTO
```

Explicación

✓ Mejor: Agregar store=True

```
total_commission = fields.Float(  
    compute='_compute_total',  
    store=True, # Ahora las búsquedas son rápidas  
)
```

Explicación

✗ Patrón 3: No usar self con iteración

```
# MALO: Falla con múltiples registros  
@api.depends('amount')  
def _compute_commission(self):  
    self.commission = self.amount * 0.1  
    # Error si self tiene 2+ registros
```

Explicación

✓ Mejor: Usar for loop

```
@api.depends('amount')  
def _compute_commission(self):  
    for record in self:  
        record.commission = record.amount * 0.1
```

Explicación

5. Pruebas de Migración y Actualización

¿Qué son y por qué hacerlas?

Estas pruebas verifican que el módulo pueda actualizarse sin perder datos o romper funcionalidad existente. También confirman que sea compatible con la versión de Odoo que usan.

¿Por qué es crítico? Los clientes acumulan datos valiosos. Una actualización que borre o corrompa datos es catastrófica.

¿Cuándo hacerlas? Siempre que reciban una actualización del módulo.

Proceso de prueba de actualización

Paso 1: Preparar ambiente de prueba

1. Hacer backup de la base de datos de producción
2. Restaurar el backup en un ambiente de pruebas
3. Confirmar que la versión vieja del módulo funciona en el backup

Paso 2: Actualizar el módulo

1. Reemplazar los archivos del módulo con la nueva versión
2. Actualizar desde la interfaz: Aplicaciones > Actualizar Lista de Aplicaciones
3. Buscar el módulo y hacer clic en 'Actualizar'
4. Revisar el log cuidadosamente durante la actualización

Paso 3: Verificar integridad de datos

- Abrir registros existentes - verificar que todos los campos tengan sus valores
- Contar registros antes y después - no debería haber pérdida de datos
- Probar cálculos automáticos - confirmar que usen nuevas reglas si aplica
- Generar reportes con datos históricos - verificar consistencia

Paso 4: Scripts de migración

Si la actualización incluye cambios en la estructura de datos, debe haber un script de migración. Verificar:

```

module_name/
├── migrations/
│   └── 16.0.2.0.0/
│       ├── pre-migrate.py    # Se ejecuta ANTES de actualizar
│       └── post-migrate.py   # Se ejecuta DESPUÉS de actualizar

```



Ejemplo de script post-migrate.py:

```

def migrate(cr, version):
    # Recalcular comisiones con nueva fórmula
    cr.execute("""
        UPDATE sale_order
        SET commission_amount = amount_untaxed * co
mmission_rate / 100
        WHERE commission_amount IS NULL
    """)

```

Explicacion



Señal de alerta:

Si la actualización cambia campos o modelos pero NO incluye scripts de migración, pueden perder datos.

6. Análisis de Logs y Debugging

¿Qué son y por qué hacerlos?

Los logs de Odoo contienen información invaluable sobre lo que está pasando internamente. Como desarrollador, debes saber interpretarlos para identificar problemas antes de que se conviertan en fallas.

¿Cuándo hacerlo? Durante todas las pruebas y especialmente si algo parece funcionar 'raro'.

Qué buscar en los logs

Tipo de Mensaje	Qué Significa	Acción Requerida
ERROR	Algo falló y no se completó la operación	CRÍTICO - Debe corregirse
WARNING	Situación anormal pero no bloqueante	Investigar - Puede causar problemas después
DeprecationWarning	Uso de funciones obsoletas que se eliminarán	Pedir al desarrollador que actualice el código
AccessError	Usuario intenta acceder a algo sin permisos	Revisar configuración de seguridad
MissingError	Referencia a registro que no existe	Verificar integridad referencial y constraints

Comandos útiles para revisar logs:

```
# Ver últimas líneas del log en tiempo real
tail -f /var/log/odoo/odoo-server.log

# Buscar errores relacionados con tu módulo
grep -i 'module_name' /var/log/odoo/odoo-server.log | grep ERROR

# Ver solo warnings
grep WARNING /var/log/odoo/odoo-server.log

# Buscar queries lentas
grep 'query took' /var/log/odoo/odoo-server.log
```

7. Checklist Técnico Final para Desarrolladores

Antes de aprobar el módulo para producción:

✓	Revisión Técnica Completada
□	Code review completo con búsqueda de anti-patrones

✓	Revisión Técnica Completada
☐	<code>__manifest__.py</code> correctamente configurado con todas las dependencias
☐	Todos los modelos tienen permisos definidos en <code>ir.model.access.csv</code>
☐	Todos los métodos sobrescritos llaman <code>super()</code>
☐	Campos computados usan <code>@api.depends</code> correctamente
☐	Campos usados en búsquedas tienen <code>store=True</code>
☐	No hay N+1 queries problems en loops
☐	Vistas XML no causan conflictos con módulos existentes
☐	Pruebas unitarias incluidas y pasando (si aplica)
☐	Análisis de performance: < 200 queries para operaciones comunes
☐	Logs limpios: sin ERRORS, mínimos WARNINGS
☐	Probado en copia de producción con datos reales
☐	Script de migración incluido si hay cambios de estructura
☐	Compatible con versión de Odoo en producción
☐	Documentación técnica para futuros mantenimientos

Conclusiones y Recomendaciones

Importancia de un Proceso Estructurado

Implementar este protocolo de pruebas puede parecer mucho trabajo inicialmente, pero es una inversión que se paga sola. Los beneficios incluyen:

- **Menos problemas en producción:** Detectar errores antes de que afecten a los clientes
- **Ahorro de tiempo:** Debuggear en producción es 10x más difícil que en desarrollo
- **Mejor relación con desarrolladores:** Feedback claro sobre qué necesita mejorarse
- **Conocimiento acumulado:** El equipo aprende y mejora con cada revisión
- **Protección del negocio:** Menos riesgos de pérdida de datos o tiempo de inactividad

Estrategia de Implementación

No intenten implementar todo de golpe. Sugerimos este enfoque gradual:

Fase 1 - Básico (Semana 1-2):

- Implementar revisión de documentación
- Pruebas funcionales básicas
- Verificación de permisos básicos
- Revisar logs por errores obvios

Fase 2 - Intermedio (Semana 3-4):

- Agregar pruebas de casos extremos
- Pruebas de compatibilidad con módulos principales
- Code review básico de estructura
- Pruebas con volumen realista de datos

Fase 3 - Avanzado (Mes 2+):

- Code review completo con análisis de performance
- Exigir pruebas unitarias para módulos complejos
- Pruebas de migración sistematizadas
- Análisis detallado de logs y queries

Comunicación con Desarrolladores Externos

Para aprovechar al máximo este documento, compartan con sus desarrolladores:

1. **Al inicio del proyecto:** Los estándares de calidad que esperan (este documento completo)
2. **Durante el desarrollo:** Checkpoints intermedios para revisar progreso
3. **En la entrega:** Los checklists específicos que usarán para evaluar
4. **Después de pruebas:** Feedback constructivo y específico sobre qué corregir

Inversión en Capacitación

Como empresa joven, consideren:

- Entrenar a al menos un desarrollador junior interno en los fundamentos de Odoo
- Capacitar a consultores en aspectos técnicos básicos (leer logs, entender estructura de módulos)
- Crear un repositorio interno de 'lecciones aprendidas' de cada revisión
- Documentar casos específicos de su industria/clientes

Palabras Finales

La calidad del software no es accidente - es el resultado de procesos estructurados y consistentes. Este documento es su guía, pero la efectividad real vendrá de:

- **Consistencia:** Usar este protocolo en TODOS los módulos, sin excepciones
- **Mejora continua:** Actualizar el documento basándose en experiencias reales
- **Cultura de calidad:** Hacer que todos entiendan POR QUÉ estas pruebas son importantes

Sunname Technologies ahora tiene las herramientas para asegurar que cada módulo que implementan sea de la más alta calidad. ¡Éxito en sus proyectos!